

The $M \times N$ Integration Problem, Worked Out by Hand

Why connecting M apps to N tools costs $M \times N$ bespoke adapters — and how one shared protocol collapses the count to $M + N$.

What this document does. It takes the one tiny scenario from Module 2.6 — the 4 *agent frameworks* (LangChain, CrewAI, AutoGen, Custom) wiring up to 4 *tool services* (GitHub, Slack, Postgres, Notion) — and counts the integrations both ways. Without a shared protocol every framework writes its own adapter for every service: $4 \times 4 = 16$ connectors. With MCP each side implements the protocol once: $4 + 4 = 8$. We then derive the general law, the savings ratio $R = \frac{MN}{M+N}$, and the marginal cost of adding one more tool or one more app. Nothing is hand-waved. Every number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent. The module quotes the headline “ $M \times N \rightarrow M + N$ ”; we rebuild it from first principles and then show what it becomes at 10×20 and 50×50 .

Where this sits. This is **Topic 1 of Module 2.6** — the combinatorial argument behind the module’s “Why MCP Matters” before/after diagram. Its companions: Module 2.5 (tools as `@tool` functions — the very things being integrated here) and the MCP course (JSON-RPC framing, transports, server lifecycle — the protocol whose existence makes the $M + N$ count possible).

Concepts covered, in order: integration as a bipartite count · the quadratic mesh $M \times N$ · the linear hub $M + N$ · the savings ratio $R = \frac{MN}{M+N}$ · why $R \approx M/2$ when $M \approx N$ · marginal cost of a new tool (M vs. 1) and a new app (N vs. 1) · MCP as the “USB-C” standardization · what stays invariant.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: four apps, four tools, small enough to count by hand

An *integration* is one piece of glue code: the adapter that lets a specific app speak to a specific tool’s specific API. In the world *before* a shared protocol, that glue is bespoke — LangChain’s GitHub adapter is not CrewAI’s GitHub adapter, and neither is reusable. Each of the M apps must write, test, and maintain its own connector to each of the N tools.

The module’s own diagram fixes the tiny example: $M = 4$ frameworks on the left, $N = 4$ services on the right. Draw every framework-to-service line and you draw the full bipartite graph $K_{M,N}$. That is the whole story — the rest is counting edges.

The quantities. We give every count a symbol. These are illustrative, picked round so you can follow the sums on paper. The mechanics are identical at any size — only M and N change.

Quantity	Symbol	Value here
AI apps / frameworks (the clients) (LangChain, CrewAI, AutoGen, Custom)	M	4
Tools / data-sources (the servers) (GitHub, Slack, Postgres, Notion)	N	4
Bespoke integrations, old world	$I_{\text{old}} = M N$	16
Protocol implementations, MCP world	$I_{\text{new}} = M + N$	8
Savings ratio	$R = I_{\text{old}}/I_{\text{new}}$	2.0

Notation. $I_{\text{old}}(M, N)$ is the number of distinct connectors to build and maintain with no shared protocol; $I_{\text{new}}(M, N)$ is the number once every app implements one MCP *client* and every tool implements one MCP *server*. R is how many times fewer pieces of glue the protocol buys you.

Intuition

Think of power adapters before USB-C. Every laptop (M) needed its own cable for every device (N): a wall of incompatible bricks, $M \times N$ of them. USB-C is a *protocol*: each laptop grows one port, each device grows one port, and any laptop talks to any device. The wall of bricks ($M \times N$) becomes a short list of ports ($M + N$). MCP is exactly that move for AI tools — it turns a dense mesh into a hub.

2 Step 1 — counting the old world: a quadratic mesh

With no shared protocol, an integration exists for *every* (app, tool) pair. The set of pairs is the Cartesian product of the M apps with the N tools, so the count is the product:

$$I_{\text{old}}(M, N) = \underbrace{M}_{\text{apps}} \times \underbrace{N}_{\text{tools}}. \quad (1)$$

This is the number of edges in the complete bipartite graph $K_{M,N}$: each of the M left nodes connects to all N right nodes. For the module's example, $I_{\text{old}}(4, 4) = 4 \times 4 = 16$.

Mini-example — this operation in isolation

Hold the apps fixed and read across one tool. *GitHub* alone needs a LangChain adapter, a CrewAI adapter, an AutoGen adapter, and a Custom adapter — 4 separate connectors for one service, none shared. Four tools $\times$ four adapters each = 16. Every cell of the 4×4 grid is a distinct codebase someone has to keep alive as GitHub's API drifts.

The damage is that this grows as a *product*. When M and N scale together it is quadratic: double both and you quadruple the integrations. That is the “quadratic explosion” the module's red diagram warns about.

3 Step 2 — counting the MCP world: a linear hub

A shared protocol inserts a standard interface in the middle. Now an app does not integrate with *tools*; it integrates with *the protocol*, once. Likewise a tool integrates with the protocol, once. The total number of implementations is therefore a *sum*, not a product:

$$I_{\text{new}}(M, N) = \underbrace{M}_{\text{one client each}} + \underbrace{N}_{\text{one server each}}. \quad (2)$$

Each app ships one MCP *client*; each tool ships one MCP *server*; the protocol is the hub they all speak through. For the module's example, $I_{\text{new}}(4, 4) = 4 + 4 = 8$ — the eight spokes of the green hub-and-spoke diagram. Already half of 16.

The savings ratio is the quotient of the two counts:

$$R(M, N) = \frac{I_{\text{old}}}{I_{\text{new}}} = \frac{M N}{M + N} \quad (3)$$

Check the module example: $R(4, 4) = \frac{16}{8} = 2.0$. ✓ The mesh costs exactly twice the hub at 4×4 — and, as the next step shows, the gap only widens.

Intuition

The product $M \cdot N$ and the sum $M + N$ start close for tiny numbers but diverge fast: at 2×2 they are equal ($4 = 4$), at 4×4 the product is double the sum, at 50×50 it is $25 \times$ the sum. A protocol does not shave a constant off the cost — it changes the *shape* of the cost from a product to a sum.

4 Step 3 — the savings ratio when apps and tools balance

How big does R get? The clean case is a balanced ecosystem, $M = N$. Substitute into Equation (3):

$$R(M, M) = \frac{M \cdot M}{M + M} = \frac{M^2}{2M} = \frac{M}{2}. \quad (4)$$

So when apps and tools grow together, **the protocol's leverage grows linearly — about half the number of participants on one side**. Ten apps and ten tools: $R = 10/2 = 5 \times$ fewer. Fifty and fifty: $R = 50/2 = 25 \times$ fewer. The bigger the ecosystem, the more the hub wins.

Mini-example — this operation in isolation

$M = N = 50$. Old world: $50 \times 50 = 2500$ bespoke integrations — an unmaintainable wall. MCP world: $50 + 50 = 100$ implementations. Ratio $R = 2500/100 = 25$, and indeed $M/2 = 50/2 = 25$. ✓ The same fifty tools that would have demanded 2500 hand-written adapters now demand 50 servers, each written *once* and consumed by all fifty apps for free.

The two headline worked numbers from this topic:

Scenario	M	N	$I_{\text{old}} = MN$	$I_{\text{new}} = M+N$	R
Module diagram	4	4	16	8	$2.0 \times$
Mid-size shop	10	20	200	30	$6.7 \times$
Balanced ecosystem	50	50	2500	100	$25.0 \times$

Sample check, 10×20 : $I_{\text{old}} = 10 \cdot 20 = 200$, $I_{\text{new}} = 10 + 20 = 30$, $R = 200/30 = 6.\bar{6} \approx 6.7$. ✓ Thirty implementations replace two hundred — the same $6.7 \times$ “fewer” the headline promises.

5 Step 4 — the marginal cost: what one more tool costs

The killer feature is not today’s count; it is what tomorrow’s growth costs. Add one new tool (raise N to $N + 1$) and ask how many *new* integrations each world incurs:

$$\Delta_{\text{tool}}^{\text{old}} = I_{\text{old}}(M, N+1) - I_{\text{old}}(M, N) = M(N+1) - MN = \boxed{M}, \quad (5)$$

$$\Delta_{\text{tool}}^{\text{new}} = I_{\text{new}}(M, N+1) - I_{\text{new}}(M, N) = (M+N+1) - (M+N) = \boxed{1}. \quad (6)$$

In the old world, every new tool must be wired into all M existing apps — a fresh adapter per app. In the MCP world, the tool author writes *one* server and every app already speaks to it. By symmetry, adding one new app costs N in the old world (it must integrate every existing tool) and 1 in the MCP world (it ships one client):

$$\Delta_{\text{app}}^{\text{old}} = N, \quad \Delta_{\text{app}}^{\text{new}} = 1.$$

Mini-example — this operation in isolation

Start at $M = 10$ apps, $N = 20$ tools ($I_{\text{old}} = 200$, $I_{\text{new}} = 30$). Ship a 21st tool. Old world: +10 new adapters (one per app), total 210. MCP world: +1 server, total 31. The marginal ratio is a full $10\times$ — and it *grows* with M . The more popular the ecosystem, the more punishing every new connector is without a protocol, and the cheaper it is with one. ✓

Watch out

The marginal cost is the real argument, not the snapshot. A team that compares only today’s totals (200 vs. 30) underweights the protocol; the decisive figure is that each future tool costs them M adapters forever versus 1 server once. In a growing ecosystem the old world’s *per-addition* cost rises without bound (it is M , and M keeps climbing), while MCP’s stays pinned at 1. That is why “we’ll just write the adapter ourselves” quietly becomes the most expensive sentence in tool engineering.

6 Step 5 — the full table, mesh vs. hub

Holding the structure fixed and only varying (M, N) , Equations (1)–(3) give the whole landscape:

M	N	$I_{\text{old}} = MN$	$I_{\text{new}} = M+N$	$R = MN/(M+N)$
2	2	4	4	1.00×
4	4	16	8	2.00×
10	10	100	20	5.00×
10	20	200	30	6.67×
20	30	600	50	12.00×
50	50	2,500	100	25.00×
100	100	10,000	200	50.00×

Read down the balanced rows (2, 4, 10, 50, 100) and R is exactly half of M every time — the $R = M/2$ law of Equation (4), in a column. At 2×2 the protocol breaks even ($R = 1$): with two apps and two tools, four adapters and four implementations cost the same, so MCP buys nothing but uniformity. From there on the hub pulls away, and by 100×100 it is doing the work of 10,000 adapters with 200 implementations — $50\times$ leverage. Sample check, 20×30 : $I_{\text{old}} = 600$, $I_{\text{new}} = 50$, $R = 600/50 = 12.0$. ✓

Intuition

There is a crossover, and it is at $M = N = 2$. Below the ecosystem's interesting size, a protocol is overhead. The instant you have more than a couple of each, the sum beats the product — and the margin compounds. Standards feel like bureaucracy when there are two of everything and like oxygen when there are fifty. The math says both feelings are correct.

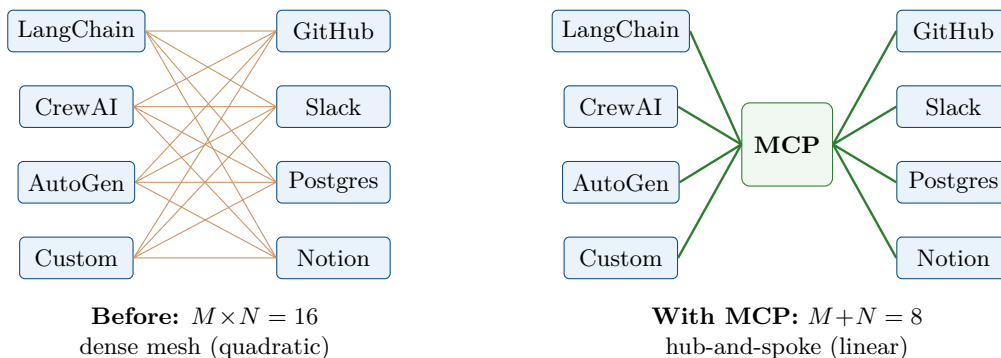
7 What this changes — and what it does not

The protocol changes the *shape*. Going from $M \cdot N$ to $M + N$ is not a discount on a fixed cost; it is a change of growth law, product $\rightarrow$ sum. That is why the savings *ratio* itself grows with the ecosystem instead of staying constant.

It moves the secrets boundary too. Because each tool is integrated once, as a server, the credentials (API keys, OAuth tokens, DB strings) live in that one server process — not copied into M separate app-side adapters. Collapsing $M \times N$ to $M + N$ also collapses M copies of each secret to 1.

What never changes: the protocol does not make a tool do more, and it does not remove the work of writing *one* good server or *one* good client. It removes the *redundant multiplication* of that work across every pair. As long as apps and tools both grow, the old world carries an $M \cdot N$ term and the MCP world carries only $M + N$. You cannot make either side disappear; you can choose whether they multiply or add.

8 The accounting, drawn



Left: every app wires to every tool — 16 edges, the complete bipartite graph $K_{4,4}$. Right: every app and every tool wires once to the protocol hub — 8 spokes. Same connectivity (any app can reach any tool), a quarter of the glue per added participant. The picture *is* Equations (1) and (2).

9 Integration cheat-sheet

Old-world integrations (no protocol)	$I_{\text{old}}(M, N) = M N$
MCP-world implementations	$I_{\text{new}}(M, N) = M + N$
Savings ratio	$R(M, N) = \frac{M N}{M + N}$
Balanced ecosystem ($M = N$)	$R = \frac{M}{2}$
Marginal cost of one new tool	old = M , MCP = 1
Marginal cost of one new app	old = N , MCP = 1
Break-even point	$M = N = 2 \Rightarrow R = 1$
Dominant growth	product $M \cdot N$ vs. sum $M + N$

10 An honest word about these numbers

The counts here ($M = N = 4$, then 10×20 and 50×50) are illustrative round numbers, picked so the products and sums are followable by hand — exactly as the module’s diagram used a neat 4×4 grid no real ecosystem stops at. Real ecosystems are lopsided and messy: there are far more tools than agent frameworks, some adapters are partly reusable, and a “protocol implementation” is not always the same size of work as a “bespoke adapter.” Those details move the constants. What is *not* illustrative is the structure: with no shared interface the integration count is the *product* $M \cdot N$, and with one it is the *sum* $M + N$, full stop. Change the numbers and the ratios move; the product-versus-sum shape does not. That shape — and its corollary that each new tool costs M adapters or 1 server — is the thing to carry into every later module about tooling, ecosystems, and why MCP exists at all.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the headline counts, the savings ratios, the marginal costs, and the full mesh-vs-hub table; change the `cases` list to explore your own ecosystem’s economics.

```

1  # mxn_integration.py -- reproduces every number in
2  # "The M x N Integration Problem, by Hand".
3
4  def old_world(M, N): # bespoke: every app re-implements every connector
5      return M * N # = edges of complete bipartite graph K_{M,N}
6
7  def mcp_world(M, N): # one MCP client per app + one MCP server per tool
8      return M + N
9
10 def ratio(M, N):
11     return old_world(M, N) / mcp_world(M, N)
12
13 # --- the module's ground example: 4 frameworks x 4 services ---
14 print("Module example 4x4:")
15 print(f" old={old_world(4,4)} new={mcp_world(4,4)} R={ratio(4,4):.4f}")
16
17 # --- the two headline worked scenarios ---
18 print("\nWorked scenarios:")
19 for (M, N) in [(10, 20), (50, 50)]:

```

```

20     print(f" M={M:2d} N={N:2d} old={old_world(M,N):5d} "
21           f"new={mcp_world(M,N):3d} R={ratio(M,N):.4f}x")
22
23 # --- R = M/2 when M = N ---
24 print("\nBalanced ecosystem R should equal M/2:")
25 for M in (2, 4, 10, 50, 100):
26     print(f" M=N={M:3d} R={ratio(M,M):6.2f} M/2={M/2:6.2f} "
27           f"match={abs(ratio(M,M)-M/2) < 1e-9}")
28
29 # --- marginal cost of one more tool / one more app ---
30 M, N = 10, 20
31 d_tool_old = old_world(M, N+1) - old_world(M, N) # = M
32 d_tool_new = mcp_world(M, N+1) - mcp_world(M, N) # = 1
33 d_app_old = old_world(M+1, N) - old_world(M, N) # = N
34 d_app_new = mcp_world(M+1, N) - mcp_world(M, N) # = 1
35 print(f"\nMarginal cost at M={M}, N={N}:")
36 print(f" +1 tool: old +{d_tool_old} (=M) mcp +{d_tool_new}")
37 print(f" +1 app : old +{d_app_old} (=N) mcp +{d_app_new}")
38
39 # --- the full mesh-vs-hub table ---
40 print("\nMesh vs hub table:")
41 cases = [(2,2),(4,4),(10,10),(10,20),(20,30),(50,50),(100,100)]
42 for (M, N) in cases:
43     print(f" M={M:3d} N={N:3d} old={old_world(M,N):6d} "
44           f"new={mcp_world(M,N):4d} R={ratio(M,N):6.2f}x")
45
46 # Expected output:
47 # Module example 4x4:
48 # old=16 new=8 R=2.0000
49 #
50 # Worked scenarios:
51 # M=10 N=20 old= 200 new= 30 R=6.6667x
52 # M=50 N=50 old= 2500 new=100 R=25.0000x
53 #
54 # Balanced ecosystem R should equal M/2:
55 # M=N= 2 R= 1.00 M/2= 1.00 match=True
56 # M=N= 4 R= 2.00 M/2= 2.00 match=True
57 # M=N= 10 R= 5.00 M/2= 5.00 match=True
58 # M=N= 50 R= 25.00 M/2= 25.00 match=True
59 # M=N=100 R= 50.00 M/2= 50.00 match=True
60 #
61 # Marginal cost at M=10, N=20:
62 # +1 tool: old +10 (=M) mcp +1
63 # +1 app : old +20 (=N) mcp +1
64 #
65 # Mesh vs hub table:
66 # M= 2 N= 2 old= 4 new= 4 R= 1.00x
67 # M= 4 N= 4 old= 16 new= 8 R= 2.00x
68 # M= 10 N= 10 old= 100 new= 20 R= 5.00x
69 # M= 10 N= 20 old= 200 new= 30 R= 6.67x
70 # M= 20 N= 30 old= 600 new= 50 R= 12.00x
71 # M= 50 N= 50 old= 2500 new= 100 R= 25.00x
72 # M=100 N=100 old= 10000 new= 200 R= 50.00x

```

— end of document —